PrimeRx Case Study: Code & Analytical Approach

This document outlines the step-by-step approach and the exact Python (Pandas) code used to solve each of the 7 business questions in the PrimeRx Commercial Opportunity Analysis case study.

---

0. Data Loading and Normalization

Approach: Before answering any questions, we must load the three Excel sheets into Pandas DataFrames. A critical data cleaning step is to normalize the `HCP_ID` columns by stripping whitespace and converting to uppercase to ensure the data merges properly. We also convert the Excel serial dates to proper `datetime` objects for accurate time-window calculations.

```
import pandas as pd

# Load data from the Excel workbook
FILE = "PrimeRx Datasets.xlsx"
alerts  = pd.read_excel(FILE, sheet_name="Alerts")
sales   = pd.read_excel(FILE, sheet_name="Sales")
affil   = pd.read_excel(FILE, sheet_name="Affiliation")

# Normalize HCP_ID to ensure reliable merging
alerts["HCP_ID"] = alerts["HCP_ID"].astype(str).str.strip().str.upper()
sales["HCP_ID"]  = sales["HCP_ID"].astype(str).str.strip().str.upper()
affil["HCP_ID"]  = affil["HCP_ID"].astype(str).str.strip().str.upper()

# Ensure dates are proper datetime objects
alerts["Alert_Date"]       = pd.to_datetime(alerts["Alert_Date"])
sales["Prescription_Date"] = pd.to_datetime(sales["Prescription_Date"])

# Define the universe of HCPs (doctors who received at least one alert)
alert_hcps = set(alerts["HCP_ID"].unique())
```

Question 1: Who are the most active doctors?

Approach: We need two separate rankings. For **Ranking 1 (Alert Volume)**, we group the `alerts` dataset by `HCP_ID` and count the rows. For **Ranking 2 (Prescription Volume)**, we filter the `sales` dataset to only include doctors from our alert universe, group by `HCP_ID`, and sum the `Prescription_Volume`.

```
# Ranking 1 - Top 10 by Alert Volume
q1_alert = (alerts.groupby("HCP_ID").size()
            .reset_index(name="Total_Alerts")
            .sort_values("Total_Alerts", ascending=False)
            .head(10).reset_index(drop=True))
q1_alert.index = q1_alert.index + 1  # 1-indexed ranks
```

```
# Ranking 2 - Top 10 by Prescription Volume (filtered to alerts universe)
sales_alert_universe = sales[sales["HCP_ID"].isin(alert_hcps)]
q1_rx = (sales_alert_universe.groupby("HCP_ID")["Prescription_Volume"]
         .sum()
         .reset_index(name="Total_Rx_Volume")
         .sort_values("Total_Rx_Volume", ascending=False)
         .head(10).reset_index(drop=True))
q1_rx.index = q1_rx.index + 1
```

Question 2: Which doctors see the need but don't act on it?

Approach: We first filter for `Positive` alerts. For each doctor, we find their *earliest* positive alert date. We then check the `sales` dataset to see if they wrote *any* prescription within a strict 30-day window following that first alert. Doctors who did not write any prescriptions are filtered out, sorted by their total positive alert count, and ranked.

```
# All HCPs with at least one Positive alert
pos_alerts = alerts[alerts["Lab_Result"] == "Positive"]

# Get the first positive alert date and total positive count per HCP
first_pos = pos_alerts.groupby("HCP_ID")
["Alert_Date"].min().reset_index(name="First_Positive_Date")
pos_count =
pos_alerts.groupby("HCP_ID").size().reset_index(name="Positive_Alert_Count")
hcp_pos = first_pos.merge(pos_count, on="HCP_ID")

# Function to check for ANY prescription within 30 days after first positive alert
def has_rx_within_30d(row):
    hcp = row["HCP_ID"]
    start = row["First_Positive_Date"]
    end   = start + pd.Timedelta(days=30)

    mask  = ((sales["HCP_ID"] == hcp) &
             (sales["Prescription_Date"] >= start) &
             (sales["Prescription_Date"] <= end))
    return sales.loc[mask, "Prescription_Volume"].sum() > 0

hcp_pos["Has_Rx_30d"] = hcp_pos.apply(has_rx_within_30d, axis=1)

# Filter for HCPs with ZERO Rx in the 30-day window
no_rx_hcps = (hcp_pos[hcp_pos["Has_Rx_30d"] == False]
              .sort_values("Positive_Alert_Count", ascending=False)
              .head(10).reset_index(drop=True))
```

Question 3: Prescription mix for Top 10 alert-volume HCPs

Approach: Taking the top 10 HCPs from Q1, we filter their sales records and categorize each sale as either "Our Drug" (`DRG001`) or "Competitor". We then pivot the data to calculate the percentage share for each category per HCP. We also group by `Drug_ID` to identify which specific competitor is stealing the most volume.

```
# Filter sales to only the Top 10 HCPs by alert volume
top10_alert_hcps = q1_alert["HCP_ID"].tolist()
top10_sales = sales[sales["HCP_ID"].isin(top10_alert_hcps)].copy()

# Categorize as Our Drug vs Competitor
top10_sales["Is_Our_Drug"] = top10_sales["Drug_ID"] == "DRG001"

# Pivot the data to get volumes side-by-side
q3_pivot = (top10_sales.groupby(["HCP_ID", "Is_Our_Drug"])["Prescription_Volume"]
            .sum().unstack(fill_value=0))
q3_pivot.columns = ["Competitor_Volume", "Our_Drug_Volume"]
q3_pivot["Total_Volume"] = q3_pivot["Our_Drug_Volume"] +
q3_pivot["Competitor_Volume"]

# Calculate percentage shares
q3_pivot["Our_Drug_%"] = (q3_pivot["Our_Drug_Volume"] / q3_pivot["Total_Volume"] *
100)
q3_pivot["Competitor_%"] = (q3_pivot["Competitor_Volume"] /
q3_pivot["Total_Volume"] * 100)

# Identify the top overall competitor drug
comp_sales = top10_sales[top10_sales["Drug_ID"] != "DRG001"]
top_comp = (comp_sales.groupby(["Drug_Name", "Drug_ID"])["Prescription_Volume"]
            .sum().reset_index(name="Total_Volume")
            .sort_values("Total_Volume", ascending=False))
```

Question 4: Top 10 accounts by conversion ratio

Approach: We shift from doctor-level to account-level analysis. We `merge` the alerts dataset with the affiliation dataset to roll up alert counts by hospital. Then we merge DRG001 sales with affiliation to roll up prescription volume by hospital. Finally, we divide prescription volume by alert count to calculate the Conversion Ratio.

```
# Roll up alerts to Account level
alerts_acc = alerts.merge(affil[["HCP_ID", "Account_ID", "Account_Name"]],
on="HCP_ID", how="inner")
acc_alerts = (alerts_acc.groupby(["Account_ID", "Account_Name"]).size()
              .reset_index(name="Total_Alert_Count")
              .sort_values("Total_Alert_Count", ascending=False))

# Take the top 10 accounts by alert volume
top10_acc = acc_alerts.head(10).copy()
```

```
# Roll up DRG001 sales to Account level
sales_drg001 = sales[sales["Drug_ID"] == "DRG001"]
sales_acc = sales_drg001.merge(affil[["HCP_ID", "Account_ID"]], on="HCP_ID",
how="inner")
acc_rx = (sales_acc.groupby("Account_ID")["Prescription_Volume"]
          .sum().reset_index(name="DRG001_Rx_Count"))

# Merge and calculate Conversion Ratio (Rx Volume / Total Alerts)
top10_acc = top10_acc.merge(acc_rx, on="Account_ID", how="left")
top10_acc["DRG001_Rx_Count"] = top10_acc["DRG001_Rx_Count"].fillna(0).astype(int)
top10_acc["Conversion_Ratio"] = top10_acc["DRG001_Rx_Count"] /
top10_acc["Total_Alert_Count"]

# Sort to find the least efficient accounts (lowest conversion ratio)
top10_acc = top10_acc.sort_values("Conversion_Ratio", ascending=True)
```

Question 5: Account size vs Rx-per-active-doctor ratio

Approach: For each account, we calculate the total number of affiliated doctors, the number of *active* doctors (those who prescribed DRG001 at least once), and the total DRG001 prescription volume. We then divide the volume by the number of active doctors to find out how productive the prescribing doctors are at each hospital.

```
# 1. Total doctors per account
acc_doc_count = (affil.groupby(["Account_ID", "Account_Name"])["HCP_ID"]
                 .nunique().reset_index(name="Total_Doctors"))

# 2. Active prescribers of DRG001 per account
drg001_prescribers = sales_drg001.merge(affil[["HCP_ID", "Account_ID"]],
on="HCP_ID", how="inner")
acc_active = (drg001_prescribers.groupby("Account_ID")["HCP_ID"]
              .nunique().reset_index(name="Active_Prescribers"))

# 3. Total DRG001 Rx volume per account
acc_vol = (drg001_prescribers.groupby("Account_ID")["Prescription_Volume"]
           .sum().reset_index(name="Total_DRG001_Rx"))

# Merge all three metrics together
q5 = acc_doc_count.merge(acc_active, on="Account_ID", how="left").merge(acc_vol,
on="Account_ID", how="left")
q5["Active_Prescribers"] = q5["Active_Prescribers"].fillna(0).astype(int)
q5["Total_DRG001_Rx"]    = q5["Total_DRG001_Rx"].fillna(0).astype(int)

# Filter out accounts with 0 active prescribers to avoid division by zero
q5_active = q5[q5["Active_Prescribers"] > 0].copy()

# Calculate ratio and rank
q5_active["Rx_Per_Active_Doctor"] = q5_active["Total_DRG001_Rx"] /
q5_active["Active_Prescribers"]
```

```
q5_bottom10 = (q5_active.sort_values("Rx_Per_Active_Doctor", ascending=True)
               .head(10).reset_index(drop=True))
```

Question 6: Behavioral Lift & Segmentation

Approach: We calculate the `Lift %` for each doctor. We define a 90-day window *before* their first positive alert and a 90-day window *after*. By aggregating DRG001 volume in both windows, we calculate the lift: `(Post - Pre) / Pre`. To handle division by zero (when Pre=0), we classify those specific doctors as "New Starters".

```
# Get first positive alert date for all HCPs
first_pos_all = pos_alerts.groupby("HCP_ID")
["Alert_Date"].min().reset_index(name="First_Positive_Date")

results = []
for _, row in first_pos_all.iterrows():
    hcp = row["HCP_ID"]
    fp  = row["First_Positive_Date"]

    # Define the 90-day windows
    pre_start  = fp - pd.Timedelta(days=90)
    pre_end    = fp
    post_start = fp
    post_end   = fp + pd.Timedelta(days=90)

    # Filter sales for this HCP
    hcp_sales = sales_drg001[sales_drg001["HCP_ID"] == hcp]

    # Sum volumes in the respective windows
    pre_vol = hcp_sales[(hcp_sales["Prescription_Date"] >= pre_start) &
                        (hcp_sales["Prescription_Date"] < pre_end)]
["Prescription_Volume"].sum()
    post_vol = hcp_sales[(hcp_sales["Prescription_Date"] > post_start) &
                         (hcp_sales["Prescription_Date"] <= post_end)]
["Prescription_Volume"].sum()

    # Segmentation Logic
    if pre_vol == 0 and post_vol > 0:
        lift_pct = None  # Cannot divide by zero
        segment  = "New Starter"
    elif pre_vol == 0 and post_vol == 0:
        lift_pct = 0.0
        segment  = "Non-responder"
    else:
        lift_pct = ((post_vol - pre_vol) / pre_vol) * 100
        segment  = "Grower" if lift_pct >= 20.0 else "Non-responder"

    results.append({
        "HCP_ID": hcp, "Pre_90d_Rx": pre_vol, "Post_90d_Rx": post_vol,
        "Lift_%": lift_pct, "Segment": segment
    })
```

```
q6 = pd.DataFrame(results)

# Create summary table for the segments
seg_summary = q6.groupby("Segment").agg(
    HCP_Count=("HCP_ID", "count"),
    Total_Pre_Rx=("Pre_90d_Rx", "sum"),
    Total_Post_Rx=("Post_90d_Rx", "sum")
).reset_index()
```

Question 7: Field Force Allocation (Capacity Math)

Approach: We calculate total field force capacity based on reps, calls per day, and field days per month over a quarter. We then conceptually map percentages (50%, 35%, 15%) to the segments derived in Q6 based on ROI.

```
# Total Capacity Math
reps = 40
calls_per_day = 8
field_days = 20

total_monthly_calls = reps * calls_per_day * field_days
total_quarterly_calls = total_monthly_calls * 3  # 19,200 calls

# Strategy mapping
# New Starters: 50% = 9,600 calls
# Growers:      35% = 6,720 calls
# Non-responders: 15% = 2,880 calls
```